

Intelligenza Artificiale e Laboratorio

Planning, Sistemi a Regole e Incertezza

Roberto Micalizio

a.a. 2023/2024

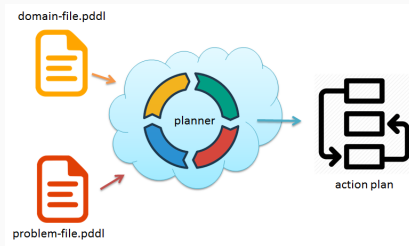
Università degli Studi di Torino,
Dipartimento di Informatica



Approfondimento sul PDDL

- **PDDL** = Planning Domain Definition Language
 - è lo standard di fatto per la definizione di domini e problemi di pianificazione usato nelle competizioni internazionali¹
- Elementi di un task di pianificazione in PDDL:
 - **Objects**: Gli oggetti che sono rilevanti per il problema considerato
 - **Predicates**: Le proprietà degli oggetti a cui siamo interessati, possono essere vere o false
 - **Initial state**: Lo stato del mondo all'inizio del problema
 - **Goal specification**: Le proprietà che devono essere vere alla fine del piano nel mondo
 - **Actions/Operators**: I modi con cui il mondo può essere modificato

¹<http://www.icaps-conference.org/index.php/Main/Competitions>



- **domain file:**
 - Predicates e Actions -> **domain knowledge** indipendente dallo specifico problema che si vuole risolvere
- **problem file:**
 - Objects, Initial state, Goal specification -> **problem knowledge** specifica per il problema di interesse

```
(define (domain <domain name>)  
  <PDDL code for predicates>  
  <PDDL code for first action>  
  [...]  
  <PDDL code for last action>  
)
```

- <domain name> è una stringa che identifica il dominio di pianificazione, es. logistics, elevators, ecc.

Problem File

```
(define (problem <problem name>)
  (:domain <domain name>)
  <PDDL code for objects>
  <PDDL code for initial state>
  <PDDL code for goal specification>
)
```

- <problem name> è una stringa che identifica il problema di pianificazione
- <domain name> identifica il dominio e deve corrispondere al nome di dominio specificato nel domain file

Gripper

In un ambiente costituito da due stanze, un robot può spostare dei palloni da una stanza ad un'altra usando i suoi due bracci. Nello stato iniziale tutti i palloni sono in una determinata stanza, il goal consiste nello spostare tutti i palloni nella seconda stanza.

- **Objects:** Le due stanze, i palloni (es. 4), i due bracci robotici
- **Predicates:** x è una stanza? x è un pallone? x è un braccio robotico?...
- **Initial state:** Tutti i palloni e il robot sono nella prima stanza, tutti i bracci robotici sono vuoti
- **Goal specification:** Tutti i palloni devono essere nella seconda stanza (nessun vincolo sulla posizione del robot)
- **Actions/Operators:** Il robot può muoversi tra le due stanze, può prendere e rilasciare un pallone con ogni braccio

PDDL by Example - Gripper DOMAIN

Predicates

- $\text{ROOM}(x)$ – true iff x è una stanza
- $\text{BALL}(x)$ – true iff x è un pallone
- $\text{GRIPPER}(x)$ – true iff x è un braccio robotico
- $\text{at-robby}(x)$ – true iff x è una stanza e il robot è al suo interno
- $\text{at-ball}(x, y)$ – true iff x è un pallone, y è una stanza e x è dentro y
- $\text{free}(x)$ – true iff x è un braccio robotico e non sta portando nulla
- $\text{carry}(x, y)$ – true iff x è un braccio robotico, y è un pallone e x sta portando y

PDDL

```
(define (domain gripper-strips
  (:predicates (ROOM ?x) (BALL ?x) (GRIPPER ?x)
    (at-robby ?x) (at-ball ?x ?y)
    (free ?x) (carry ?x ?y)
  )
```


PDDL by Example - Gripper DOMAIN

Action/Operator Move

Description: Il robot può muoversi dalla stanza x alla stanza y

Precondition: ROOM(x), ROOM(y) and at-robby(x)

Effect: positivi: at-robby(y); negativi: at-robby(x)

PDDL

```
(:action move
  :parameters (?x ?y)
  :precondition (and (ROOM ?x)
                     (ROOM ?y)
                     (at-robby ?x))
  :effect
    (and (at-robby ?y)
          (not (at-robby ?x))))
```

PDDL by Example - Gripper DOMAIN

Action/Operator Pick up

Description: Il robot può prendere un pallone x quando è nella stanza y con il braccio z

Precondition: BALL(x), ROOM(y), GRIPPER(z), at-ball(x , y), at-robby(y), free(z)

Effect: positivi: carry(z , x); negativi: at-ball(x , y), free(z)

PDDL

```
(:action pick-up
  :parameters (?x ?y ?z)
  :precondition (and (BALL ?x) (ROOM ?y) (GRIPPER ?z)
                    (at-ball ?x ?y) (at-robby ?y) (free ?z))
  :effect
    (and (carry ?z ?x)
         (not (at-ball ?x ?y)) (not (free ?z))))
```

PDDL by Example - Gripper DOMAIN

Action/Operator Drop

Description: Il robot può rilasciare un pallone x quando è nella stanza y se lo tiene con il braccio z

Precondition: BALL(x), ROOM(y), GRIPPER(z), carry(z, x), at-robby(y)

Effect: positivi: at-ball(x, y), free(z); negativi: carry(z, x))

PDDL

```
(:action drop
  :parameters (?x ?y ?z)
  :precondition (and (BALL ?x) (ROOM ?y) (GRIPPER ?z)
    (carry ?z ?x) (at-robby ?y))
  :effect
    (and (at-ball ?x ?y) (free ?z)
      (not (carry ?z ?x))))
```

PDDL by Example - Gripper DOMAIN

```
(define (domain gripper-strips)
  (:predicates (room ?r) (ball ?b) (gripper ?g) (at-robby ?r)
               (at ?b ?r) (free ?g) (carry ?o ?g))

  (:action move
    :parameters (?from ?to)
    :precondition (and (room ?from) (room ?to) (at-robby ?from))
    :effect (and (at-robby ?to) (not (at-robby ?from))))

  (:action pick
    :parameters (?obj ?room ?gripper)
    :precondition (and (ball ?obj) (room ?room) (gripper ?gripper)
                      (at ?obj ?room) (at-robby ?room) (free ?gripper))
    :effect (and (carry ?obj ?gripper) (not (at ?obj ?room))
                (not (free ?gripper))))

  (:action drop
    :parameters (?obj ?room ?gripper)
    :precondition (and (ball ?obj) (room ?room) (gripper ?gripper)
                      (carry ?obj ?gripper) (at-robby ?room))
    :effect (and (at ?obj ?room) (free ?gripper)
                (not (carry ?obj ?gripper)))))
```

gripper-domain.pddl

PDDL by Example - Gripper PROBLEM

- Quali sono gli oggetti specifici per il nostro problema?
- Quanti palloni?
- Quali stanze?
- Com'è configurato lo stato iniziale?
- Qual è il goal che si vuole raggiungere?

Objects:

- Rooms: rooma, roomb
- Balls: ball1, ball2, ball3, ball4
- Robot arms: left, right

PDDL by Example - Gripper PROBLEM

```
(define (problem strips-gripper2)
  (:domain gripper-strips)
  (:objects rooma roomb ball1 ball2 ball3 ball4 left right)
  (:init
    (room rooma) (room roomb)
    (ball ball1) (ball ball2) (ball ball3) (ball ball4)
    (gripper left) (gripper right)
    (at-robbly rooma)
    (free left)
    (free right)
    (at ball1 rooma) (at ball2 rooma)
    (at ball3 rooma) (at ball4 rooma))
  (:goal (and (at ball1 roomb) (at ball2 roomb)
    (at ball3 roomb) (at ball4 roomb))))
```

gripper-4balls.pddl

PDDL: Advanced Features

- PDDL è diventato un linguaggio complesso, ma il suo scopo rimane quello di essere uno standard per poter confrontare diversi pianificatori
- La clausola (`:requirements ...`) permette di stabilire quali segmenti del linguaggio sono usati in un particolare dominio/problema
- Un pianificatore esamina la lista dei requirements e stabilisce se è in grado di risolvere o meno il problema, o se possibilmente, considerare solo una parte “core” della definizione del problema, ignorando i segmenti non supportati

PDDL: requirements [Ghallab et al. 1998]

<i>Requirement</i>	<i>Description</i>
:strips	Basic STRIPS-style adds and deletes
:typing	Allow type names in declarations of variables
:disjunctive-preconditions	Allow or in goal descriptions
:equality	Support = as built-in predicate
:existential-preconditions	Allow exists in goal descriptions
:universal-preconditions	Allow forall in goal descriptions
:quantified-preconditions	= :existential-preconditions + :universal-preconditions
:conditional-effects	Allow when in action effects
:action-expansions	Allow actions to have :expansions
:foreach-expansions	Allow actions expansions to use foreach (implies :action-expansions)
:dag-expansions	Allow labeled subactions (implies :action-expansions)
:domain-axioms	Allow domains to have :axioms
:domain-axioms	Allow domains to have :axioms
:subgoal-through-axioms	Given axioms $p \supset q$ and goal q , generate subgoal p
:safety-constraints	Allow :safety conditions for a domain
:expression-evaluation	Support eval predicate in axioms (implies :domain-axioms)
:fluents	Support type (fluent t). Implies :expression-evaluation
:open-world	Don't make the "closed-world assumption" for all predicates — i.e., if an atomic formula is not known to be true, it is not necessarily assumed false
:true-negation	Don't handle not using negation as failure, but treat it as in first-order logic (implies :open-world)
:adl	= :strips + :typing + :disjunctive-preconditions + :equality + :quantified-preconditions + :conditional-effects
:ucpop	= :adl + :domain-axioms + :safety-constraints

PDDL requirements - Esempio

```
(define (domain hanoi-domain)
  (:requirements :equality)
  (:predicates (disk ?x) (smaller ?x ?y) (on ?x ?y) (clear ?x))
  (:action move-disk
    :parameters (?disk ?below-disk ?new-below-disk)
    :precondition
      (and (disk ?disk)
           (smaller ?disk ?new-below-disk)
           (not (= ?new-below-disk ?below-disk))
           (not (= ?new-below-disk ?disk))
           (not (= ?below-disk ?disk))
           (on ?disk ?below-disk)
           (clear ?disk)
           (clear ?new-below-disk))
    :effect
      (and (clear ?below-disk)
           (on ?disk ?new-below-disk)
           (not (on ?disk ?below-disk))
           (not (clear ?new-below-disk)))))
```

strips + equality = uso della funzione di uguaglianza nelle precondizioni

Due tipi predefiniti:

- `object`
- `number`

E' possibile definire gerarchie di tipi che abbiano uno dei due tipi predefiniti come radice.

Esempio:

```
(:types integer float - number physob)
```

Definisce `integer` e `float` come sottotipi di `number` e `physob` come sottotipo di `object` (implicito)

Richiede `(:requirements :types)`

PDDL: tipi e costanti - Esempio

```
(define (domain gripper-typed)
  (:requirements :typing)
  (:types room ball gripper)
  (:constants left right - gripper)
  (:predicates (at-robby ?r - room)
                (at ?b - ball ?r - room)
                (free ?g - gripper)
                (carry ?o - ball ?g - gripper))
  (:action move
    :parameters (?from ?to - room)
    :precondition (at-robby ?from)
    :effect (and (at-robby ?to)
                  (not (at-robby ?from))))
  (:action pick
    :parameters (?obj - ball ?room - room ?gripper - gripper)
    :precondition (and (at ?obj ?room) (at-robby ?room) (free ?gripper))
    :effect (and (carry ?obj ?gripper)
                  (not (at ?obj ?room))
                  (not (free ?gripper))))
```

PDDL: tipi e costanti - Esempio Gerarchia di Tipi

```
(define (domain logistics-adl)
  (:requirements :adl :domain-axioms)
  (:types physobj - object
           obj vehicle - phssobj
           truck airplane - vehicle
           location city - object
           airport - location)
  (:predicates (at ?x - physobj ?l - location)
               (in ?x - obj ?t - vehicle)
               (in-city ?l - location ?c - city)))
```

- (:requirements :adl :domain-axioms) sono necessari per gli effetti condizionali (vedi slide successiva)
- :types definisce una gerarchia di tipi tutti discendenti da object
- i predicati sono poi definiti in base al tipo più specifico necessario

```
(:action drive-truck
:parameters (?truck - truck ?loc-from ?loc-to - location ?city - city)
:precondition (and (at ?truck ?loc-from) (in-city ?loc-from ?city)
                  (in-city ?loc-to ?city))
:effect (and (at ?truck ?loc-to)
             (not (at ?truck ?loc-from))
             (forall (?x - obj)
               (when (and (in ?x ?truck)
                          (and (not (at ?x ?loc-from))
                              (at ?x ?loc-to)))))))
```

- gli oggetti che sono at ?loc-from e anche a bordo del truck (in ?truck) si trovano at ?loc-to